FOOD4ALL · HITO 3 · CLASE 14

Del requisito a las cuatro vistas

RF-05 · Carrito multivendedor (US-04, Must)

Proceso (DFD) · Datos (DER y modelo lógico aplicado) · Arquitectura · Interfaz (wireframe)

Campo	Valor
Versión	v0 (primera versión del taller)
Fecha	30/10/2026
Responsable	Juan Antonio (backend en el Sprint 3)
Revisor	Pendiente: un miembro distinto del autor
Requisito de origen	US-04 / RF-05 Carrito multivendedor · Must · MVP
Fuente	Línea base v0.4 del Hito 2 (repositorio sebasbarrabaja-cmyk/Food4All, docs/requisitos/) y Clase 14

1. Objetivo y requisito de origen 3
Criterios de aceptación que las vistas deben reflejar 3
Subrequisitos de RF-05 3
2. Vista 1 · Proceso (DFD) 4
Excepciones 4
3. Vista 2 · Datos 6
3.1 Modelo conceptual (DER) 6
3.2 Modelo lógico aplicado 6
3.3 DDL de referencia 7
4. Vista 3 · Arquitectura 9
Contrato de la API (propuesta v0 de backend) 9
5. Vista 4 · Interfaz (wireframe) 11
6. Control de calidad: los cuatro errores habituales 12
7. Radar: dependencias, degradación y recuperación 12
8. Trazabilidad y registro 13
Decisiones de diseño v0 13
Supuestos y dudas abiertas 13
Registro obligatorio de la Clase 14 13
9. Reglas del repositorio aplicadas 13

1. Objetivo y requisito de origen

La Clase 14 pide representar un requisito Must existente desde cuatro vistas complementarias (proceso, datos, arquitectura e interfaz) y justificar qué evidencia falta. **No se diseña a partir de intuiciones ni se añade ningún requisito:** todo sale de la línea base v0.4 del Hito 2.

US-04 / RF-05 · Carrito multivendedor. Como cliente, quiero añadir productos de varios vendedores a un solo carrito, para no hacer varios pedidos. Es el primer paso del núcleo diferencial de Food4All: varios vendedores, una compra, una entrega.

Campo	Valor
Necesidades	N-CAR-01 (comprar a varios vendedores en un solo pedido) · N-ENT-01 (recibirlo todo junto) · N-PLA-02 (planificar la compra)
Proceso anterior	RF-03 / RF-04 Catálogo: productos con precio y stock del día
Proceso siguiente	RF-06 Checkout y pago único
RNF que aplican	RNF-03 carga ≤ 2,5 s en móvil 4G (hipótesis) · RNF-04 primera compra sin ayuda · RNF-05 WCAG 2.2 AA
Actor	Cliente. La historia US-04 dice «estudiante» porque es el cliente objetivo; aquí se generaliza, como en las fichas de RF-05

Criterios de aceptación que las vistas deben reflejar

1. Dado un carrito con un producto del vendedor A, cuando añado un producto del vendedor B, entonces el carrito muestra dos grupos, A y B, y un único total (RF-05.2-01, RF-05.2-04).
2. Dado un producto con stock 0, cuando intento añadirlo, entonces el botón está desactivado y se muestra «Agotado» (RF-04.4-04).

Subrequisitos de RF-05

ID	Subrequisito	Regla principal
05.1	Añadir producto	Crea la línea o suma 1, nunca por encima del stock del día
05.2	Vista agrupada por vendedor	Un grupo por vendedor y un único total (Decidido)
05.3	Cambiar cantidad o quitar	Recálculo inmediato y «Deshacer» durante 5 s
05.4	Un solo envío y compra mínima	Un envío por pedido (Decidido); mínimo de 10–15 € (Hipótesis H-01)
05.5	Presupuesto	Avisa si se supera, nunca bloquea la compra (Decidido)
05.6	Agotado o cambio de precio	Se comprueba al abrir y al pagar; nunca se cobra un precio distinto del mostrado
05.7	Carrito guardado	En la cuenta con sesión; 7 días en el navegador sin ella
05.8	Diseño	Líneas a la izquierda (2/3) y resumen fijo con «Pagar» a la derecha (1/3)
05.9	Información de entrega	Próxima franja y por qué hay un solo envío

2. Vista 1 · Proceso (DFD)

¿Qué ocurre y quién intercambia qué información? Los cinco procesos leen o escriben el mismo carrito (D1) y tres de ellos consultan el catálogo antes de aceptar una cantidad o un precio. Cada flecha lleva el nombre del dato, nunca «datos» ni «API».

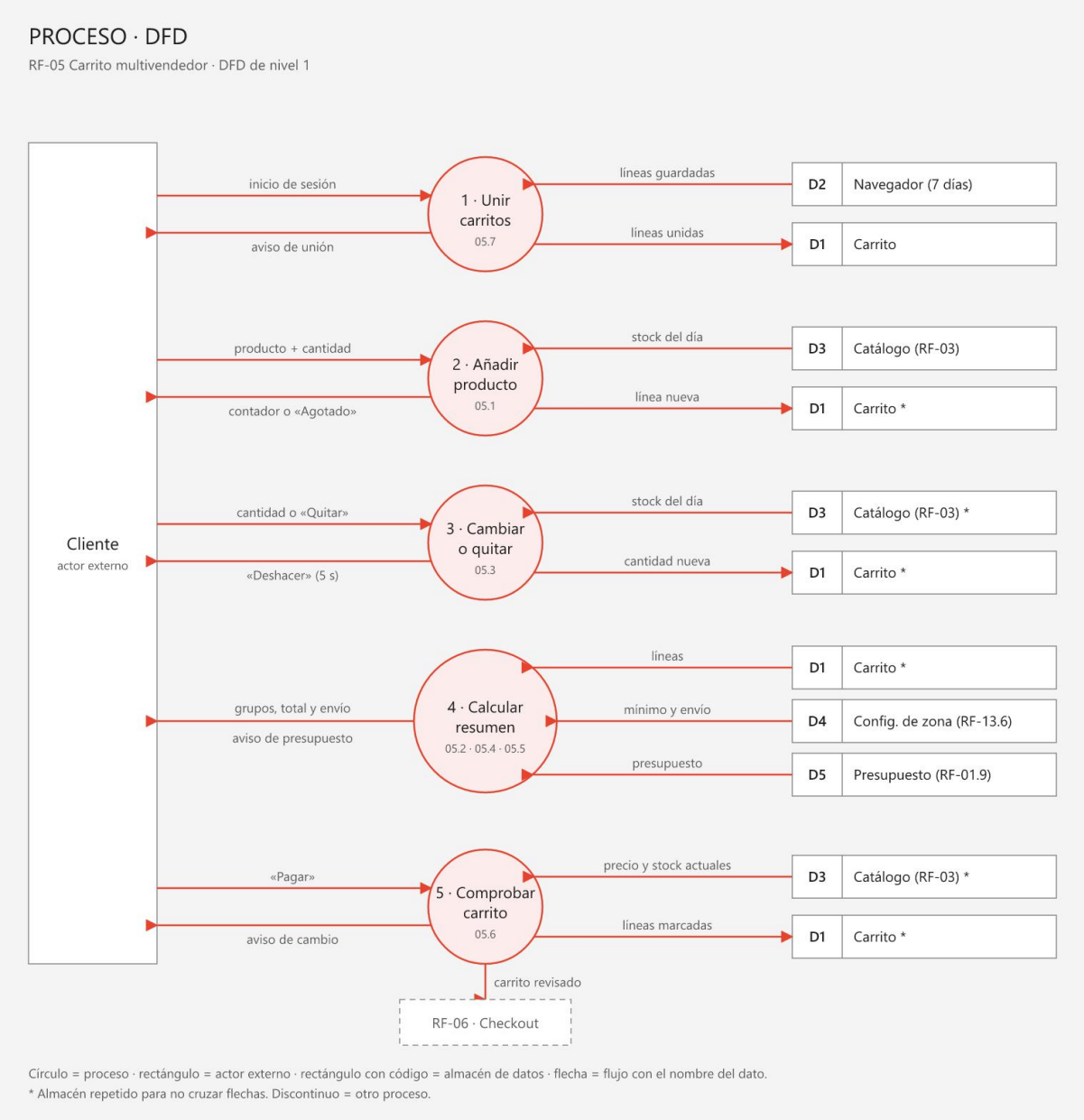


Figura 1. DFD de nivel 1 de RF-05 con la notación de la Clase 14.

El proceso 4 es el diferencial de Food4All: agrupa por vendedor y calcula un solo total con un solo envío. El proceso 5 se ejecuta al abrir el carrito y al pulsar «Pagar», y solo entrega a RF-06 un carrito sin líneas pendientes de revisar.

Excepciones

Excepción	Proceso	Qué ve el cliente	Qué se guarda
Se agota al añadir	2	«Este producto se acaba de agotar»	Nada

Excepción	Proceso	Qué ve el cliente	Qué se guarda
Cantidad por encima del stock	3	«Solo quedan <n> unidades»	La cantidad, ajustada al máximo
Se agota después de añadirlo	5	«Ya no está disponible», fuera del total	La línea, marcada
Cambia el precio	5	«El precio ha cambiado de 2,00 € a 2,20 €»	La línea, marcada
Todo agotado	5	«Los productos de tu carrito se han agotado»	Carrito vacío
No llega al mínimo	4	«Te faltan 2,00 € para el mínimo» y «Pagar» desactivado	Nada
Cambia de zona con carrito	2	Pregunta si vaciar; si lo mantiene, la zona no cambia	Nada
Navegador sin almacenamiento	1	Nada visible	El carrito solo vive con la página abierta
El catálogo no responde	2, 3 y 5	Sin definir en la línea base	Duda abierta (apartado 7)

3. Vista 2 · Datos

3.1 Modelo conceptual (DER)

¿Qué información existe y cómo se relaciona? El carrito solo guarda sus líneas. Los grupos por vendedor, los subtotales y el total se calculan en cada cambio, así nunca contradicen a las líneas.

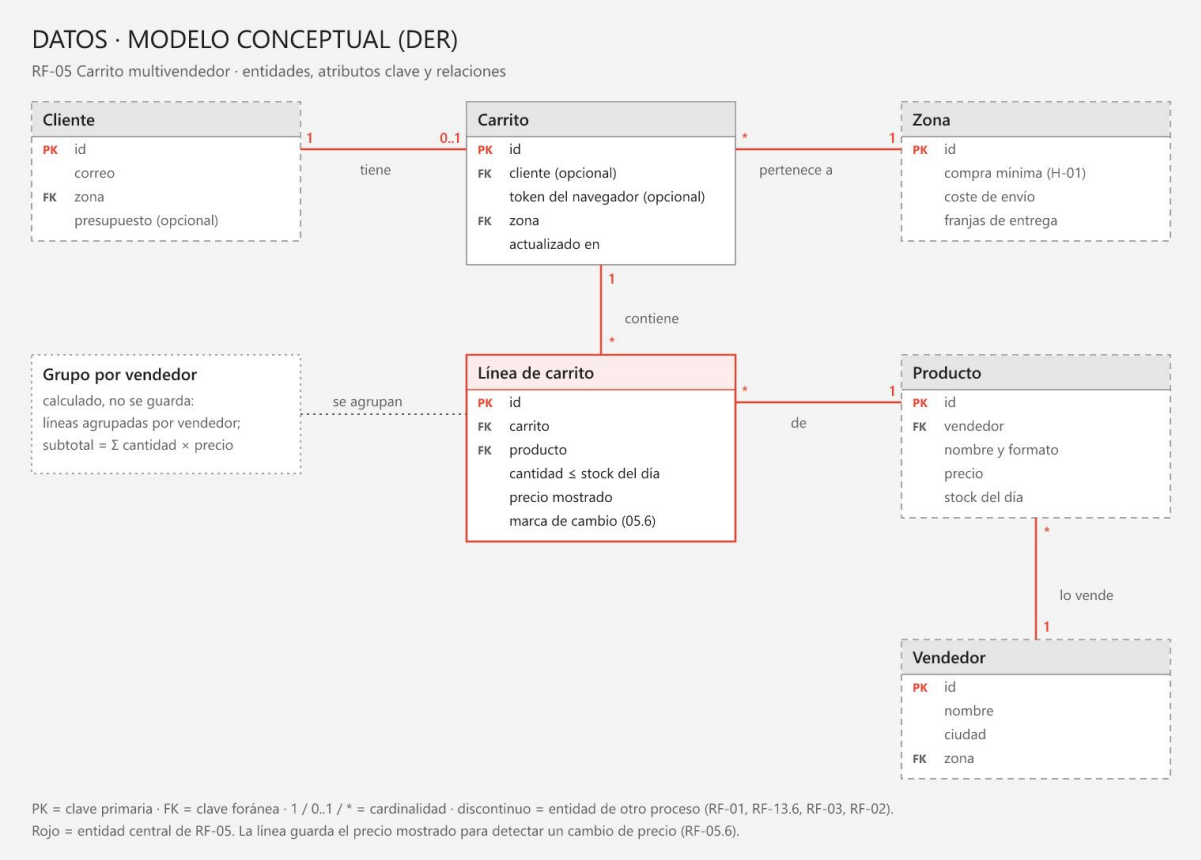


Figura 2. Modelo conceptual: entidades, atributos clave y cardinalidades.

Dato	Tipo	Dónde vive	Regla clave
Carrito	Persistente	En la cuenta con sesión; 7 días en el navegador sin ella	Una sola zona por carrito (propuesta, D-03)
Línea de carrito	Persistente	Dentro del carrito	Cantidad entre 1 y el stock del día; guarda el precio mostrado
Grupo por vendedor, subtotales y total	Calculado	Se recalcula en cada cambio (≤ 300 ms, propuesta)	No se guarda
Aviso «Deshacer»	Temporal	En pantalla durante 5 s	Al caducar, el cambio queda firme
Producto y vendedor	Externo al proceso	RF-03 y RF-02	RF-05 solo los lee
Compra mínima y envío	Externo al proceso	RF-13.6, los fija el administrador	Un cambio con el carrito abierto se aplica al recargar
Presupuesto	Externo, opcional	RF-01.9 o RF-12.1	Avisa, pero nunca bloquea la compra

3.2 Modelo lógico aplicado

Paso del DER a tablas relacionales con tipos, claves y restricciones. **Es una propuesta v0 de backend para el equipo:** el gestor de base de datos está por decidir (issue #11 «Base de datos común») y los tipos son SQL estándar.

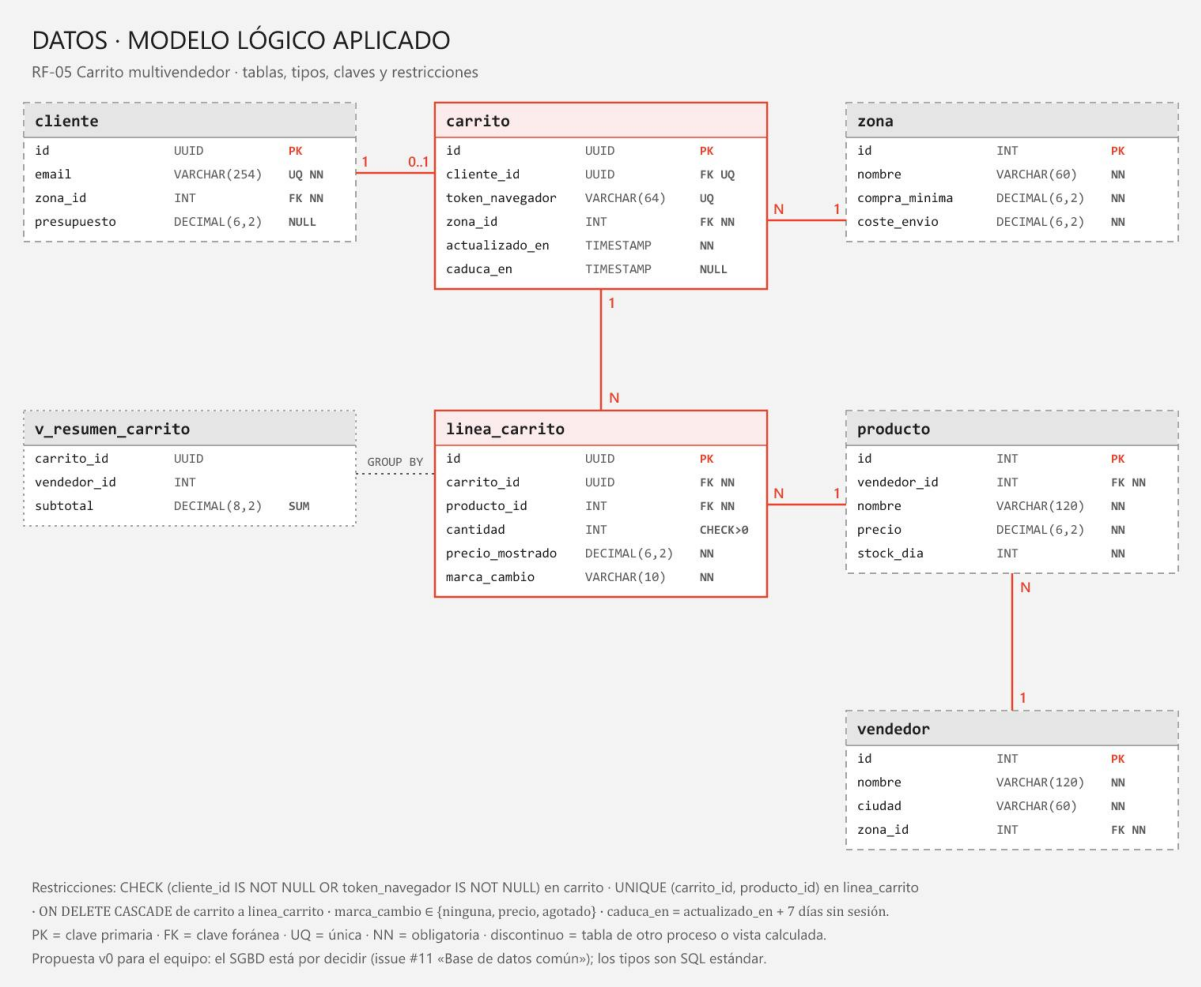


Figura 3. Modelo lógico: tablas propias de RF-05 en rojo; tablas de otros procesos y vista calculada en discontinuo.

Tabla	Clave primaria	Claves foráneas	Restricciones
carrito	id (UUID)	cliente_id → cliente.id (opcional, única) zona_id → zona.id	CHECK: cliente_id o token_navegador no nulos · token_navegador único · caduca_en = actualizado_en + 7 días sin sesión
linea_carrito	id (UUID)	carrito_id → carrito.id (ON DELETE CASCADE) producto_id → producto.id	UNIQUE (carrito_id, producto_id) · CHECK cantidad > 0 · marca_cambio ∈ {ninguna, precio, agotado}
v_resumen_carrito (vista)	—	—	SUM(cantidad × precio_mostrado) GROUP BY carrito_id, vendedor_id
cliente, zona, producto, vendedor	id	Según su proceso (RF-01, RF-13.6, RF-03, RF-02)	Solo se leen desde RF-05

3.3 DDL de referencia

```
CREATE TABLE carrito (
  id               UUID PRIMARY KEY,
```

```
  cliente_id       UUID UNIQUE REFERENCES cliente(id),
  token_navegador  VARCHAR(64) UNIQUE,
  zona_id          INT NOT NULL REFERENCES zona(id),
  actualizado_en   TIMESTAMP NOT NULL,
  caduca_en        TIMESTAMP,
  CHECK (cliente_id IS NOT NULL OR token_navegador IS NOT NULL)
);

CREATE TABLE linea_carrito (
  id               UUID PRIMARY KEY,
  carrito_id       UUID NOT NULL REFERENCES carrito(id) ON DELETE CASCADE,
  producto_id      INT NOT NULL REFERENCES producto(id),
  cantidad         INT NOT NULL CHECK (cantidad > 0),
  precio_mostrado  DECIMAL(6,2) NOT NULL,
  marca_cambio     VARCHAR(10) NOT NULL DEFAULT 'ninguna'
                   CHECK (marca_cambio IN ('ninguna', 'precio', 'agotado')),
  UNIQUE (carrito_id, producto_id)
);

CREATE VIEW v_resumen_carrito AS
  SELECT l.carrito_id, p.vendedor_id,
         SUM(l.cantidad * l.precio_mostrado) AS subtotal
  FROM linea_carrito l JOIN producto p ON p.id = l.producto_id
  WHERE l.marca_cambio <> 'agotado'
  GROUP BY l.carrito_id, p.vendedor_id;
```

La vista excluye las líneas agotadas para que el total no las cuente (RF-05.6). La línea guarda el precio mostrado: al comprobar, se compara con producto.precio y, si no coinciden, se marca «precio».

4. Vista 3 · Arquitectura

¿Qué responsabilidades colaboran para cumplirlo? El carrito no depende de ningún servicio externo a Food4All. Su dependencia crítica es interna: el catálogo (RF-03), que da el stock y el precio de cada línea.

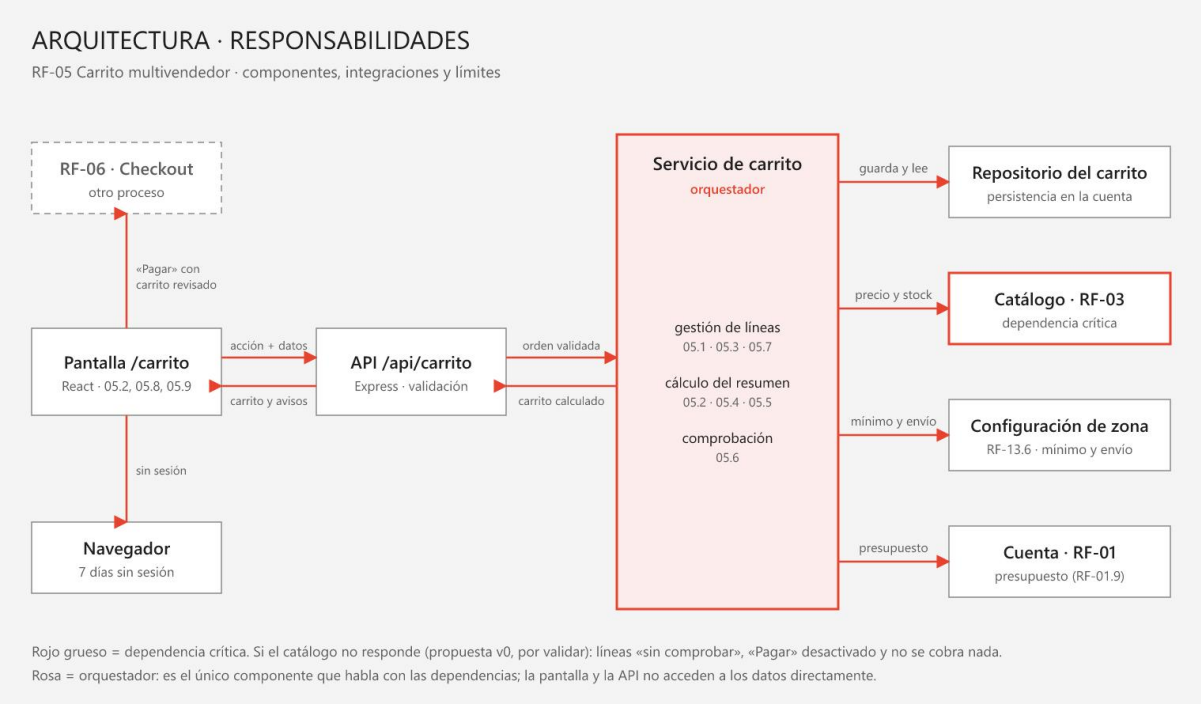


Figura 4. Arquitectura de alto nivel: el servicio de carrito orquesta y es el único que habla con las dependencias.

Componente	Responsabilidad	Requisitos
Pantalla /carrito (React)	Mostrar grupos, total, envío y avisos; enviar las acciones del cliente	RF-05.2, 05.8, 05.9
API /api/carrito (Express)	Validar la entrada, autorizar por sesión y devolver el carrito calculado	RNF-01
Servicio de carrito	Gestión de líneas, cálculo del resumen y comprobación de precio y stock	RF-05.1 a 05.7
Repositorio del carrito	Guardar y leer carrito y líneas en la cuenta	RF-05.7
Navegador	Carrito sin sesión durante 7 días	RF-05.7
Catálogo · RF-03	Precio y stock del día (dependencia crítica)	RF-05.1, 05.3, 05.6
Configuración de zona · RF-13.6	Compra mínima y coste de envío	RF-05.4
Cuenta · RF-01	Presupuesto opcional	RF-05.5

Contrato de la API (propuesta v0 de backend)

Hoy la API solo tiene el alta de vendedores y el panel. Esta es la propuesta para RF-05, a acordar con el equipo antes de programar:

Método y ruta	Entrada	Respuesta correcta	Errores

Método y ruta	Entrada	Respuesta correcta	Errores
GET /api/carrito	—	200 · carrito con grupos, totales y avisos	—
POST /api/carrito/lineas	productoId, cantidad	201 · carrito actualizado	409 «Este producto se acaba de agotar» · 422 cantidad no válida
PATCH /api/carrito/lineas/:id	cantidad (0 = quitar)	200 · carrito actualizado	422 por encima del stock: se ajusta al máximo con aviso
DELETE /api/carrito/lineas/:id	—	204	404 línea inexistente
POST /api/carrito/unir	tokenNavegador	200 · carrito unido y aviso de unión	404 token caducado
POST /api/carrito/comprobar	—	200 · líneas marcadas (precio o agotado)	503 catálogo no disponible: líneas «sin comprobar» y «Pagar» desactivado

5. Vista 4 · Interfaz (wireframe)

¿Qué ve, hace y entiende cada persona? El cliente ve de quién es cada producto, pero paga y recibe una sola vez: tres grupos, un total de 28,90 € y un envío de 3,90 €. Precios de ejemplo, meramente representativos.

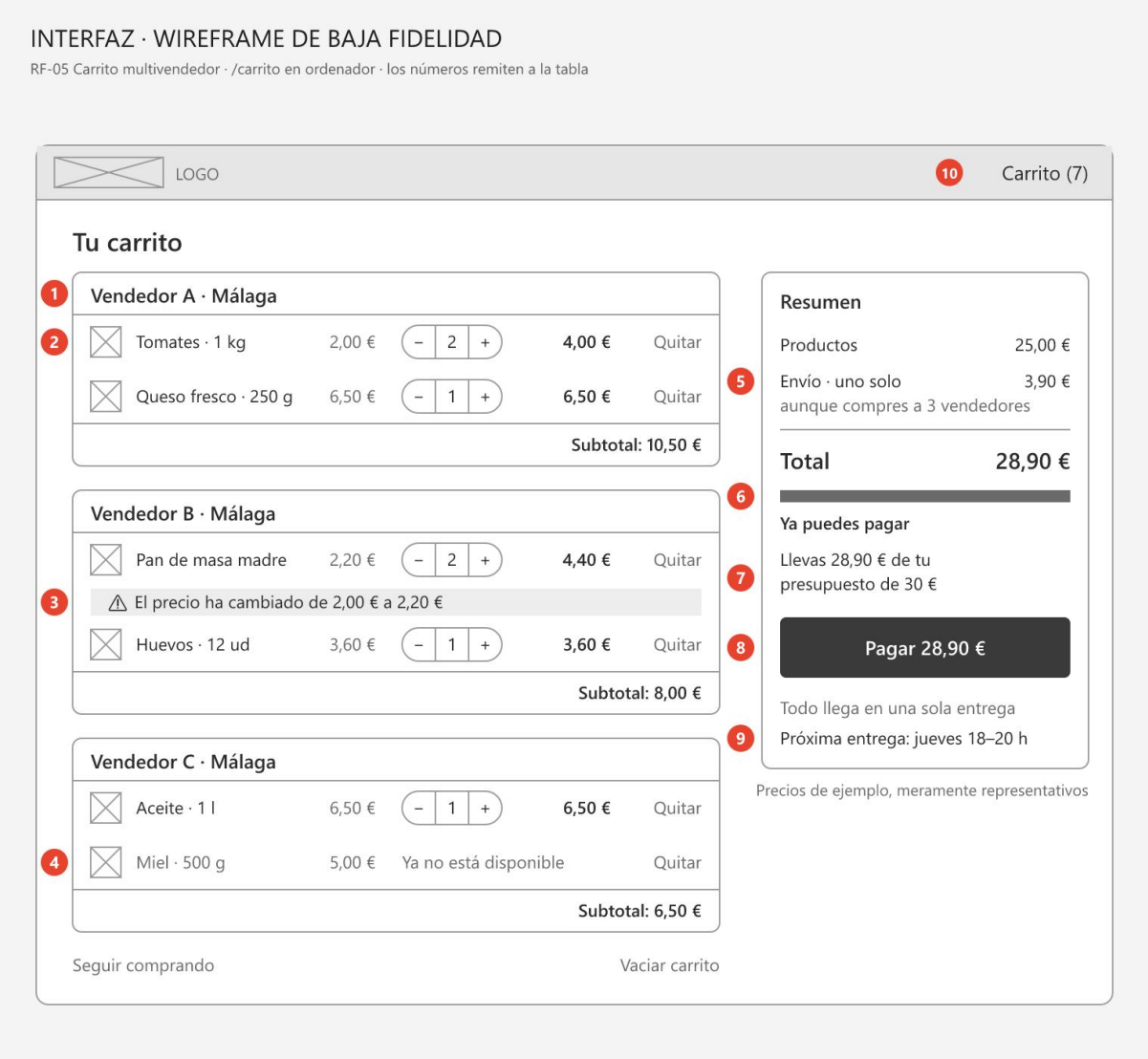


Figura 5. Wireframe de baja fidelidad de /carrito con el caso normal y las dos excepciones de RF-05.6.

Nº	Elemento	Texto exacto	Requisito	Criterio que demuestra
1	Grupo por vendedor	«Vendedor A · Málaga» y su subtotal	RF-05.2	US-04, criterio 1
2	Cantidad y quitar	«−» y «+» de 44 px; «Producto quitado. Deshacer» 5 s	RF-05.3	«+» desactivado al llegar al stock
3	Cambio de precio	«El precio ha cambiado de 2,00 € a 2,20 €»	RF-05.6	No se cobra un precio distinto del mostrado
4	Producto agotado	«Ya no está disponible», fuera del total	RF-05.6	El total no lo incluye
5	Un solo envío	«Envío: 3,90 € · uno solo aunque compres a varios vendedores»	RF-05.4	El envío se ve antes del checkout

Nº	Elemento	Texto exacto	Requisito	Criterio que demuestra
6	Compra mínima	«Te faltan 2,00 € para el mínimo» / «Ya puedes pagar»	RF-05.4	«Pagar» desactivado bajo el mínimo
7	Presupuesto	«Llevas 24,10 € de tu presupuesto de 30 €»	RF-05.5	Avisa, no bloquea
8	Botón de pago	«Pagar 28,90 €» con el importe exacto	RF-05.8	Lleva a RF-06
9	Entrega	«Próxima entrega: jueves 18–20 h»	RF-05.9	Sin franjas: «No hay franjas disponibles esta semana»
10	Contador	Unidades del carrito en la cabecera	RF-05.1	US-04, criterio 2 («Agotado» al añadir)

Estados que no salen en el dibujo: carrito vacío («Tu carrito está vacío» con «Ver productos»), «Vaciar carrito» con su confirmación y todo agotado. El diseño móvil queda para V2.

6. Control de calidad: los cuatro errores habituales

Vista	Lo que no basta	Pregunta de control	Respuesta en RF-05
Proceso	«El usuario compra» sin entradas, salidas ni excepciones	¿Qué información entra, cambia y sale?	Cada flujo lleva su dato (figura 1) y hay 9 excepciones con su mensaje
Datos	Una lista de campos sin entidad, relación ni propietario	¿Qué se guarda, durante cuánto y con qué relación?	DER con cardinalidades, modelo lógico con claves y duración (cuenta o 7 días)
Arquitectura	«Usamos nube e IA» sin responsabilidades ni dependencia	¿Quién responde si una dependencia falla?	El servicio de carrito; catálogo marcado como crítico con su modo degradado propuesto
Interfaz	Pantalla bonita sin estados, errores ni criterio	¿Qué puede hacer, entender y comprobar la persona?	10 elementos numerados, cada uno con su requisito y su criterio

7. Radar: dependencias, degradación y recuperación

El incidente de Google Cloud del 20/10/2026 (región europe-west2, 6 h 10 min sin servicio) enseña que un software correcto puede fallar si su diseño no explicita dependencias, degradación y recuperación. La clase pide a cada equipo identificar cuatro cosas:

Pregunta del radar	Respuesta para RF-05	Estado
Dependencias críticas	Catálogo (RF-03) para precio y stock; base de datos del carrito. No hay servicios de terceros	Identificado
Qué operación puede degradarse	Comprobación de precio y stock: líneas «sin comprobar» y «Pagar» desactivado; presupuesto y próxima entrega se ocultan si faltan	Propuesta v0
Qué dato necesita recuperarse	El carrito en la cuenta (líneas con su precio mostrado). El del navegador es de mejor esfuerzo: 7 días	Propuesta v0

Pregunta del radar	Respuesta para RF-05	Estado
Qué supuesto de disponibilidad queda por validar	Que el catálogo responda a tiempo para cumplir RNF-03 (LCP ≤ 2,5 s) y el recálculo en menos de 300 ms	Hipótesis

8. Trazabilidad y registro

Trazabilidad mínima: N-CAR-01 → US-04 / RF-05 → cuatro vistas v0 (figuras 1 a 5) → decisiones y supuestos de abajo → revisión por otro miembro (pendiente).

Decisiones de diseño v0

1. Los grupos por vendedor, los subtotales y el total se calculan y no se guardan, para que nunca contradigan a las líneas.
2. La línea guarda el precio mostrado para poder detectar un cambio de precio (RF-05.6).
3. El actor se llama «cliente», como en las fichas de RF-05.
4. Modelo lógico y contrato de la API: propuestas de backend, pendientes de acordar con el equipo y con el issue #11.

Supuestos y dudas abiertas

- Modo degradado si el catálogo no responde: hay que aprobarlo y registrarlo como cambio en la línea base.
- Compra mínima de 10–15 € (hipótesis H-01) y un carrito por zona (propuesta, D-03).
- Quién paga la logística (pendiente en RF-05.4).
- Recalcular en menos de 300 ms y comprobar al abrir y al pagar (propuestas de RF-05.3 y RF-05.6).
- Gestor de base de datos (issue #11).

Registro obligatorio de la Clase 14

Versión	Fecha	Responsable	Requisito	Supuesto o decisión	Fuente	Cambio
v0	30/10/2026	Juan Antonio	US-04 / RF-05	Las cuatro de arriba	Línea base v0.4 y Clase 14	Primera versión de DFD, DER, modelo lógico, arquitectura y wireframe; ningún requisito cambia

Siguiente paso: registrar las vistas en la subtarea de RF-05 en Asana (lo hace la persona, sin comentarios del asistente) y en la landing, pedir la revisión a otro miembro y contrastar alternativas de diseño para la arquitectura de alto nivel.

9. Reglas del repositorio aplicadas

- Sin datos personales ni comercios reales: los vendedores son «Vendedor A, B y C».
- Datos y precios de ejemplo marcados como tal («Precios de ejemplo, meramente representativos»).
- Hecho, hipótesis y decisión separados: cada regla lleva Decidido, Hipótesis o Propuesta.
- No se inventa evidencia ni se cambia ningún requisito decidido.
- Prompts de la sesión registrados en prompts/2026-10-30-01-sesion-cuatro-vistas-y-reglas.md.
- Redactado con ayuda del asistente; la revisión crítica la escribe la persona.